

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры

«Вычислительная техника»

"__" _____ 20__ г.

Заведующий кафедрой

_____ М.А. Митрохин

ОТЧЕТ ПО УЧЕБНОЙ (ЭКСПЛУАТАЦИОННОЙ) ПРАКТИКЕ
(2025/2026 учебный год)

Бауэр Артём Александрович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 2 семестр 4

Период прохождения практики с 19.06.26 по 16.07.26

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.

(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики д.т.н., профессор, Карамышева Н.С.

(должность, ученая степень, ученое звание)

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Утвержден на заседании кафедры

«Вычислительная техника»

" ____ " _____ 20 ____ г.

Заведующий кафедрой

_____ М.А. Митрохин

**ИНДИВИДУАЛЬНЫЙ ПЛАН ПРОХОЖДЕНИЯ УЧЕБНОЙ
(ЭКСПЛУАТАЦИОННОЙ) ПРАКТИКИ**

(2025/2026 учебный год)

_____ Бауэр Артём Александрович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4года

Год обучения _____ 2 _____ семестр _____ 4 _____

Период прохождения практики с 19.06.26 по 16.07.26

Кафедра «Вычислительная техника»

Заведующий кафедрой д.т.н., профессор, Митрохин М.А.

(должность, ученая степень, ученое звание, Ф.И.О.)

Руководитель практики д.т.н., профессор, Карамышева Н.С.

(должность, ученая степень, ученое звание)

№ п/п	Планируемая форма работы во время практики	Количество часов	Календарные сроки проведения работы	Подпись руководителя практики от вуза
1	Выбор темы и разработка индивидуального плана проведения работ	1	19.06.2026 – 19.06.2026	
2	Подбор и изучение материала по теме работы	3	19.06.2026 – 19.06.2026	
3	Установка и настройка операционной системы Linux	4	19.06.2026 – 19.06.2026	
4	Установка и настройка среды разработки (IDE)	2	20.06.2026 – 20.06.2026	
5	Разработка алгоритма	3	20.06.2026 – 20.06.2026	
6	Написание программы на языке C++	5	20.06.2026 – 20.06.2026	
5	Компиляция, сборка и отладка программы	6	20.06.2026 – 20.06.2026	
6	Тестирование программы и анализ результатов	8	21.06.2026 – 21.06.2026	
7	Оформление отчёта	10	21.06.2026 – 21.06.2026	
	Общий объём часов	42		

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЧЁТ
ОПРОХОЖДЕНИИ УЧЕБНОЙ (ЭКСПЛУАТАЦИОННОЙ) ПРАКТИКИ
(2025/2026 учебный год)

Бауэр Артём Александрович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 2 семестр 4

Период прохождения практики с 19.06.26 по 16.07.26

Кафедра «Вычислительная техника»

Бауэр А. А. выполнял практическое задание «Сортировка Шелла». На первоначальном этапе установил и настроил программу VirtualBox для создания виртуальной машины. Затем он установил на неё операционную систему Linux, после чего развернул среду разработки Geany для написания кода. Далее студент провёл детальный анализ алгоритма сортировки Шелла и его основных модификаций, после чего выбрал наиболее подходящий метод решения и определился с языком программирования C++. На основе сделанного выбора написал программу, реализующую сортировку Шелла, затем выполнил её отладку и всестороннее тестирование на различных наборах входных данных. После получения результатов провёл анализ эффективности работы программы и оформил итоговый отчёт о проделанной работе.

Бакалавр Бауэр А.А. " " _____ 2026 г.

Руководитель Карамышева Н.С. " " _____ 2026 г.
практики

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ПЕНЗЕНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ПОЛИТЕХНИЧЕСКИЙ ИНСТИТУТ
ФАКУЛЬТЕТ ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

ОТЧЁТ
ОПРОХОЖДЕНИИ УЧЕБНОЙ (ОЗНАКОМИТЕЛЬНОЙ) ПРАКТИКИ

(2025/2026 учебный год)

Бауэр Артём Александрович

Направление подготовки 09.03.01 «Информатика и вычислительная техника»

Форма обучения – очная Срок обучения в соответствии с ФГОС – 4 года

Год обучения 2 семестр 4

Период прохождения практики с 19.06.26 по 16.07.26

Кафедра «Вычислительная техника»

В процессе выполнения практики Бауэр А.А. решал следующие задачи: освоение работы с системой виртуализации VirtualBox; развёртывание операционной системы Linux на виртуальной машине; установка и настройка среды разработки Geany; углублённое изучение алгоритма сортировки Шелла и его модификаций; выбор оптимального метода решения и языка программирования C++; разработка, отладка и тестирование программы, реализующей сортировку Шелла; проведение сравнительного анализа эффективности алгоритма на разных входных данных; оформление отчёта по результатам работы.

Во время выполнения работы Бауэр А.А. показал себя ответственным, добросовестным студентом, знающим свой предмет, имеющим представление о современном состоянии науки, владеющим современными общенаучными знаниями по информатике и вычислительной технике. За выполнение работы Бауэр А.А. заслуживает оценки «_____».

Бакалавр Бауэр А.А. "___" _____ 2026 г.

Руководитель Карамышева Н.С. "___" _____ 2026 г.
практики

Оглавление

Введение.....	7
1. Установка ОС Linux на виртуальную машину	8
1.1 Установка VirtualBox.....	8
1.2 Установка виртуальной машины на VirtualBox.....	10
1.3 Установка и настройка ОС Linux Ubuntu на VirtualBox	12
1.4 ОС Linux Ubuntu.....	15
2. Установка IDE Geany.....	18
2.1 Описание процесса установки	18
2.2 Описание IDE Geany.....	19
3. Разработка программы на языке C++ в среде Geany	21
3.1 Требования к программным средствам	21
3.2 Требования к программным средствам	22
3.3 Функциональное тестирование	24
4. Анализ результатов.....	29
Заключение	29
Список используемой литературы	31
Приложение А	32

Введение

Современный мир информационных технологий немыслим без разнообразия программных платформ. Особое место среди них занимает семейство операционных систем на базе ядра Linux, которое в совокупности с утилитами проекта GNU образует мощную экосистему, известную как GNU/Linux. Данная платформа представляет собой не просто операционную систему, а целую философию открытости, безопасности и гибкости, что делает её исключительно привлекательной как для корпоративных пользователей, так и для индивидуальных разработчиков. В отличие от проприетарных решений, Linux предлагает широкий выбор дистрибутивов – от пользовательских Ubuntu и Linux Mint до серверных CentOS и Debian, каждый из которых адаптирован под конкретные задачи.

Особенно ценным качеством Linux является его ориентация на профессиональную разработку программного обеспечения.

Для целей обучения и экспериментальной работы наиболее удобным инструментом является использование виртуальных машин. Технология виртуализации позволяет эмулировать полноценный компьютер со своим процессором, оперативной памятью и дисковым пространством, работающий изолированно от основной (хостовой) системы. Благодаря этому, разработчик может развернуть несколько операционных систем на одном физическом устройстве, переключаясь между ними без необходимости перезагрузки.

Целью данной практической работы является практическое освоение процессов установки, настройки и эксплуатации операционной системы Linux в виртуальной среде, а также приобретение навыков создания и отладки прикладного программного обеспечения на языке C++ с использованием современных интегрированных сред разработки (IDE).

1. Установка ОС Linux на виртуальную машину

1.1 Установка VirtualBox

Одним из наиболее распространённых средств виртуализации является VirtualBox – кроссплатформенный продукт с дружелюбным интерфейсом, доступный для операционных систем Windows, Linux и macOS [3]. Установка выполняется стандартным способом: скачивание установочного пакета с официального сайта и запуск инсталлятора. Изображение приветственного окна представлено на рисунке 1 [2].

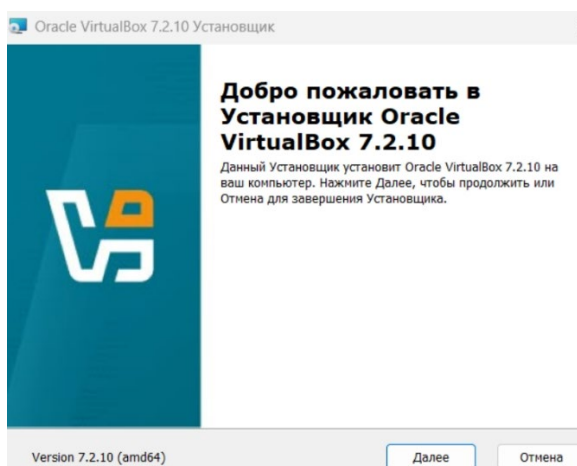


Рисунок 1 – Запуск установочного файла VirtualBox 7.2.10

Далее приняли лицензионное соглашение. Изображение окна представлено на рисунке 2.

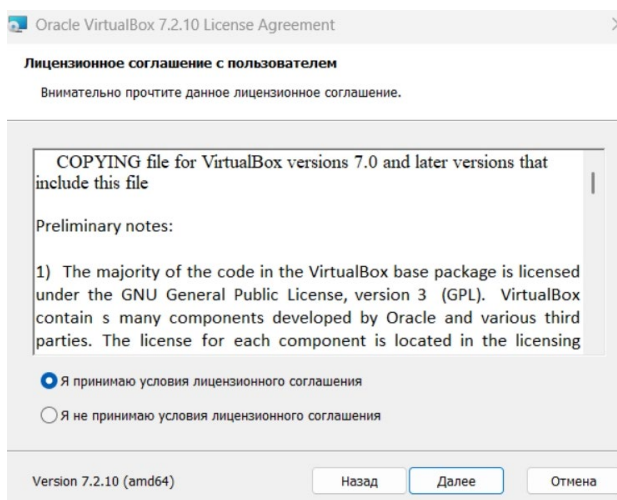


Рисунок 2 – Лицензионное соглашение

Затем выбрали компоненты для установки. Изображение представлено на рисунке 3.

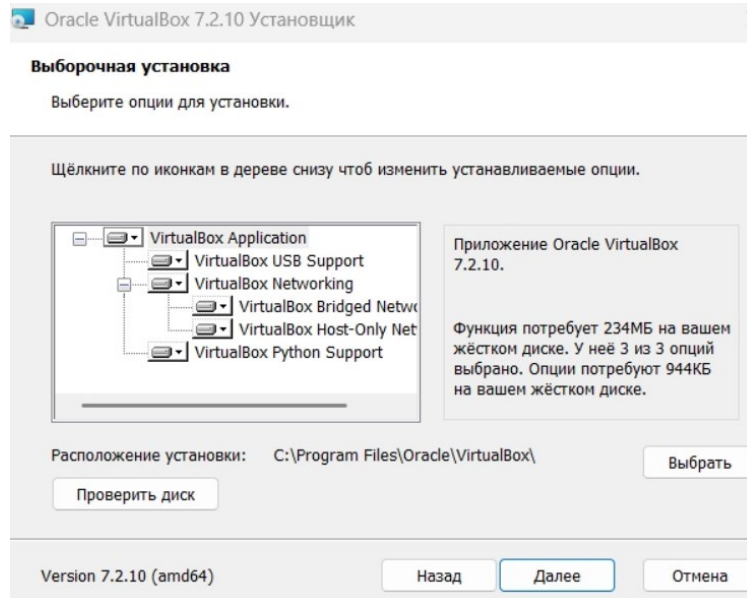


Рисунок 3 – Выбор компонентов для установки

Создали ярлык на рабочем столе. Изображение представлено на рисунке 4.

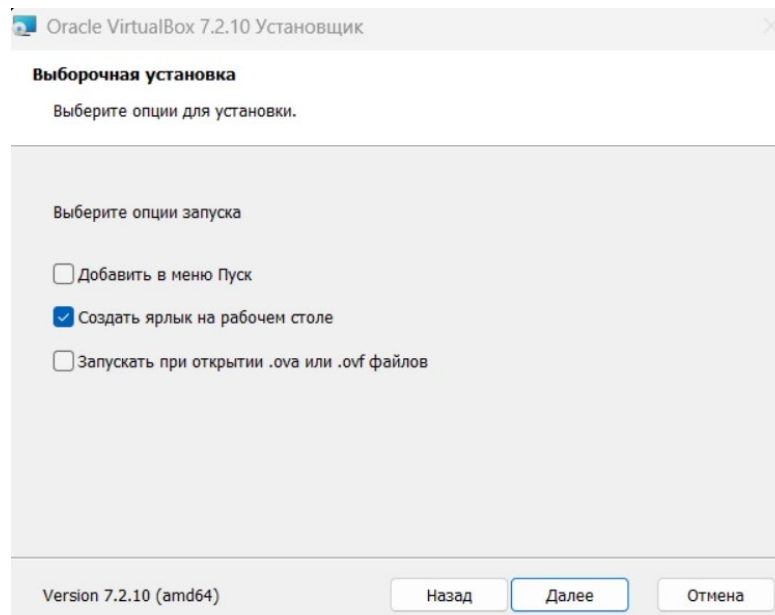


Рисунок 4 – Создание ярлыка на рабочем столе

Открыли VirtualBox. Изображение представлено на рисунке 5.

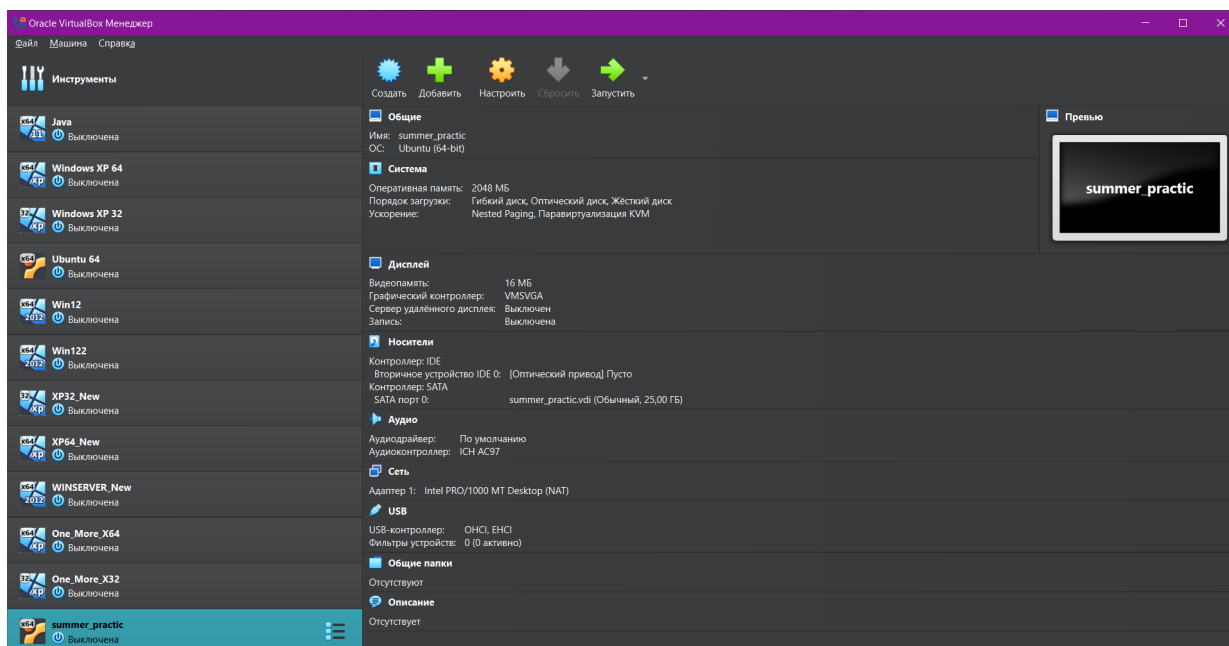


Рисунок 5 – Интерфейс VirtualBox

1.2 Установка виртуальной машины на VirtualBox

Ввели название виртуальной машины и указали путь к образу. Изображение представлено на рисунке 6.

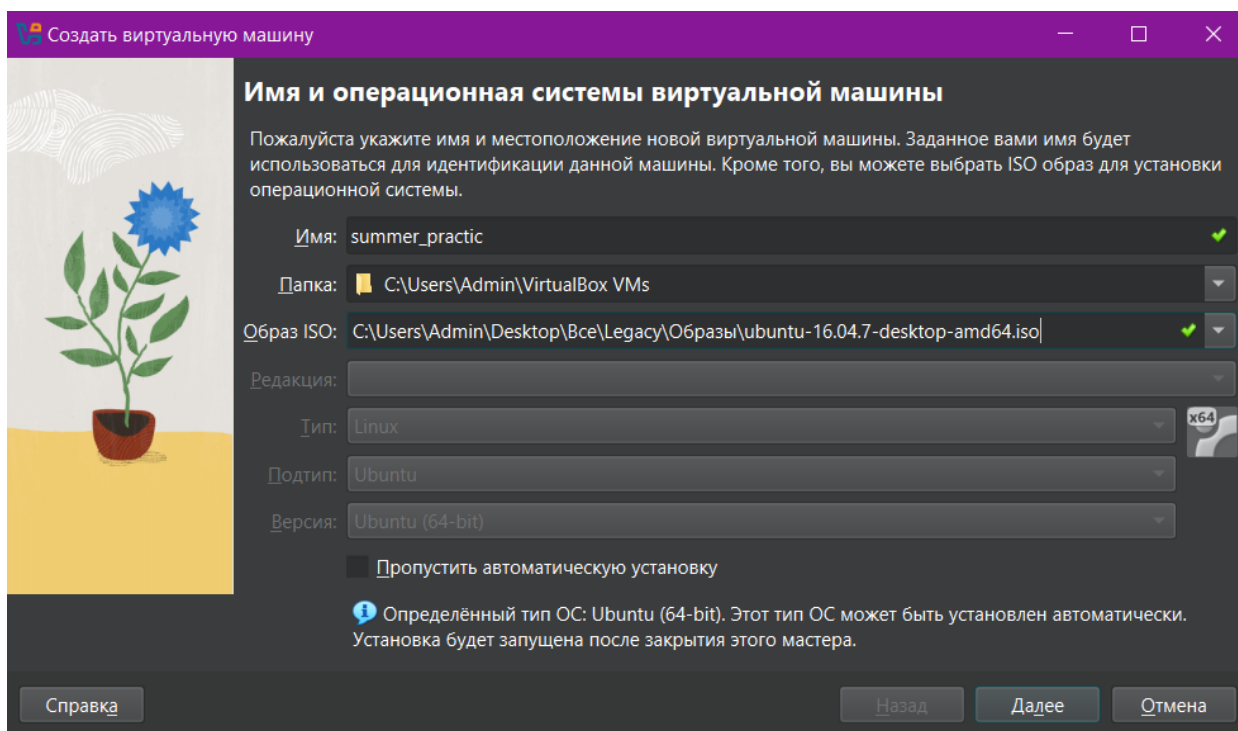


Рисунок 6 – Создание виртуальной машины

Выделили память виртуальной машине. Изображение представлено на рисунке 7.

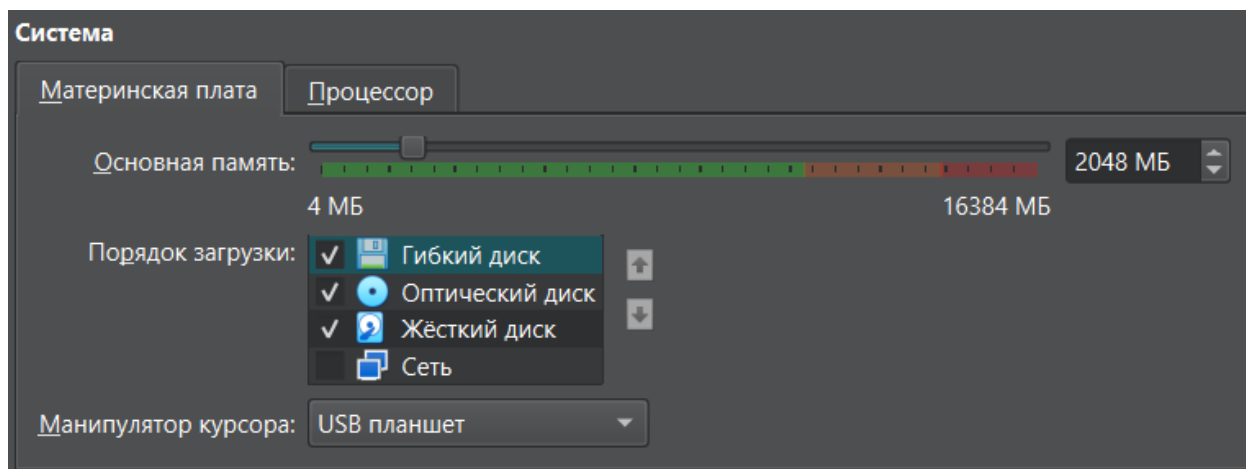


Рисунок 7 – Выделение памяти

Отредактировали остальные параметры. Изображение представлено на рисунке 8.

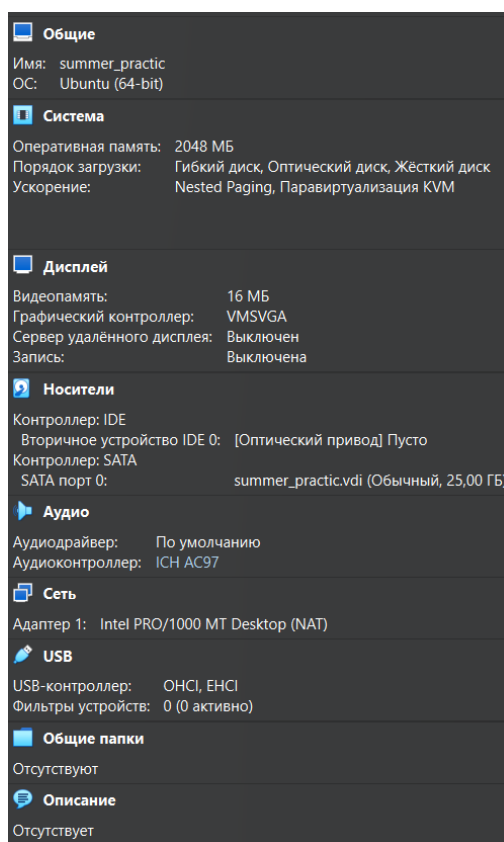


Рисунок 8 – Параметры виртуальной машины

1.3 Установка и настройка ОС Linux Ubuntu на VirtualBox

Для установки ОС Ubuntu скачали с официального сайта файл, содержащий дистрибутив операционной системы, и запустили созданную виртуальную машину «Ubuntu» [3].

Затем выбрали язык нашей ОС. Изображение представлено на рисунке 9.

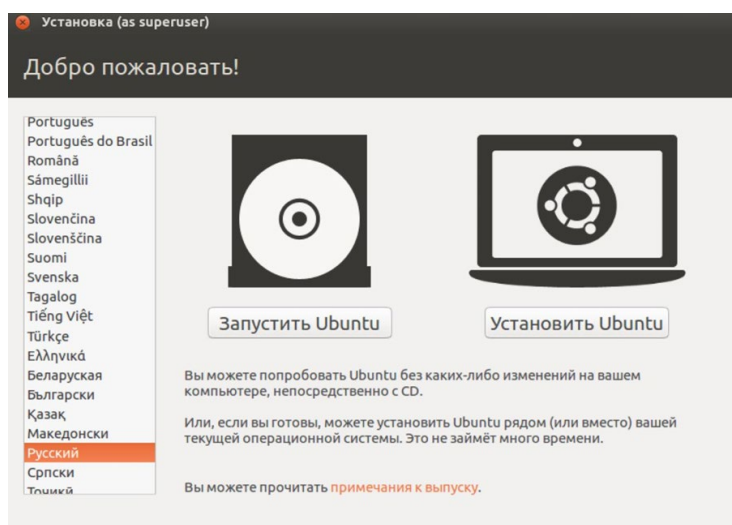


Рисунок 9 – Окно выбора языка и начала установки Ubuntu

Начали подготовку к установке. Изображение представлено на рисунке 10.

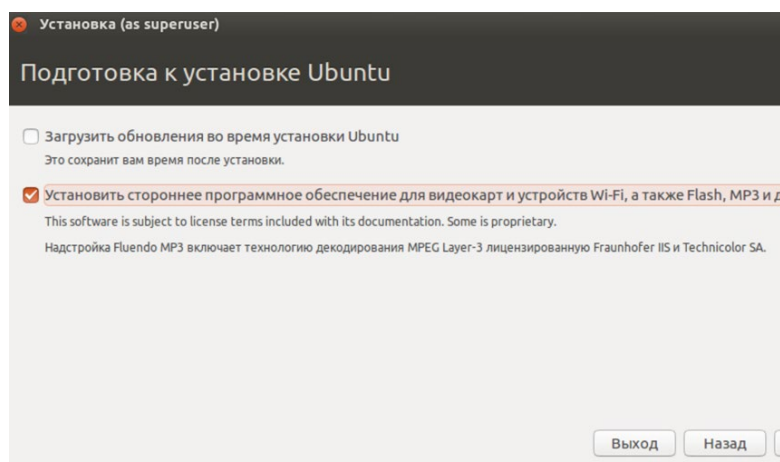


Рисунок 10 – Окно подготовки к установке

Выбрали тип установки. Изображение представлено на рисунке 11.

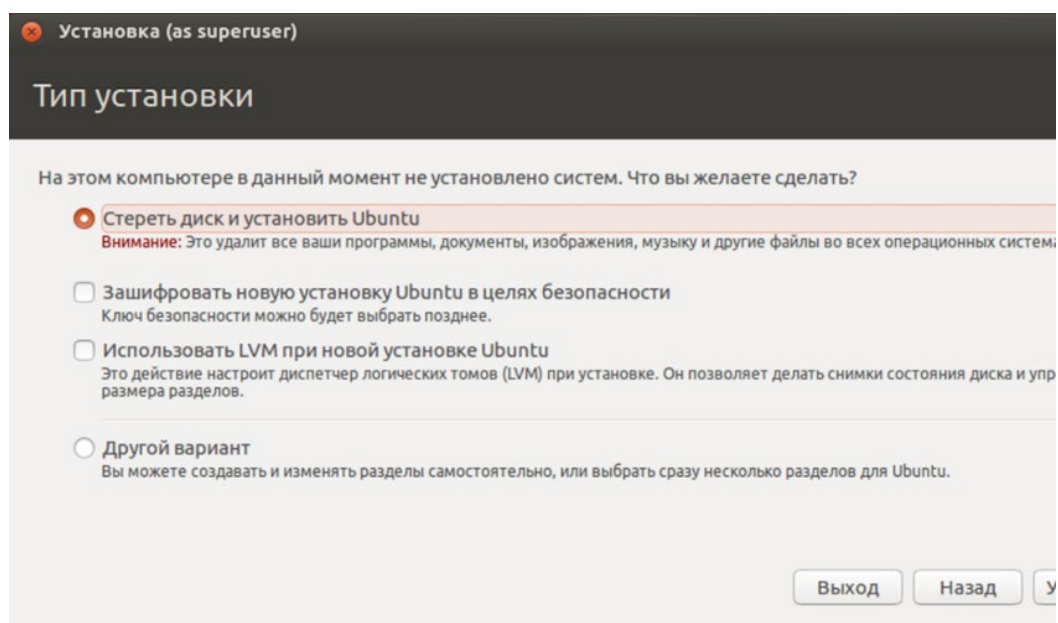


Рисунок 11 – Окно выбора типа установки

Установили регион. Изображение представлено на рисунке 12.

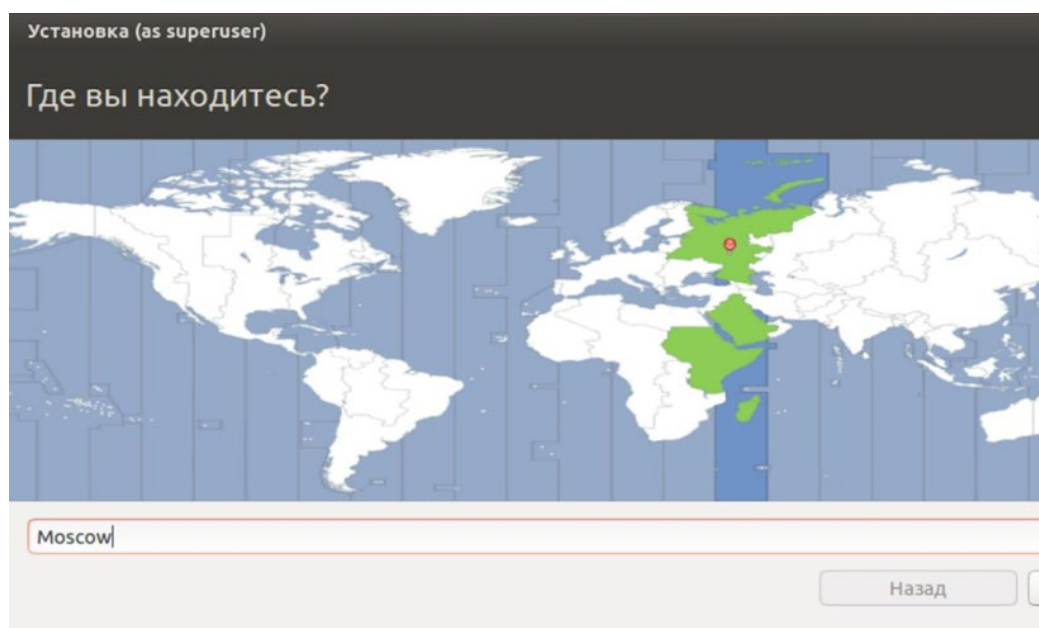


Рисунок 12 – Окно выбора региона

Установили раскладку клавиатуры. Изображение представлено на рисунке 13.

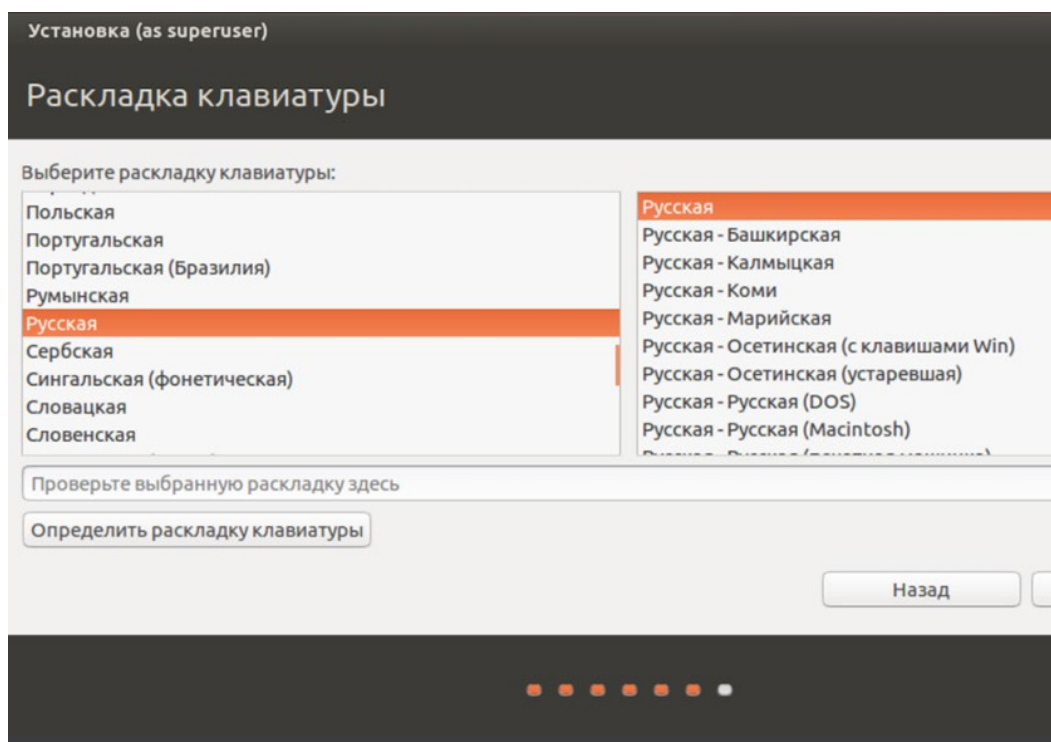


Рисунок 13 – Окно установки раскладки клавиатуры

Ввели имя компьютера, ник пользователя и пароль. Изображение представлено на рисунке 14.

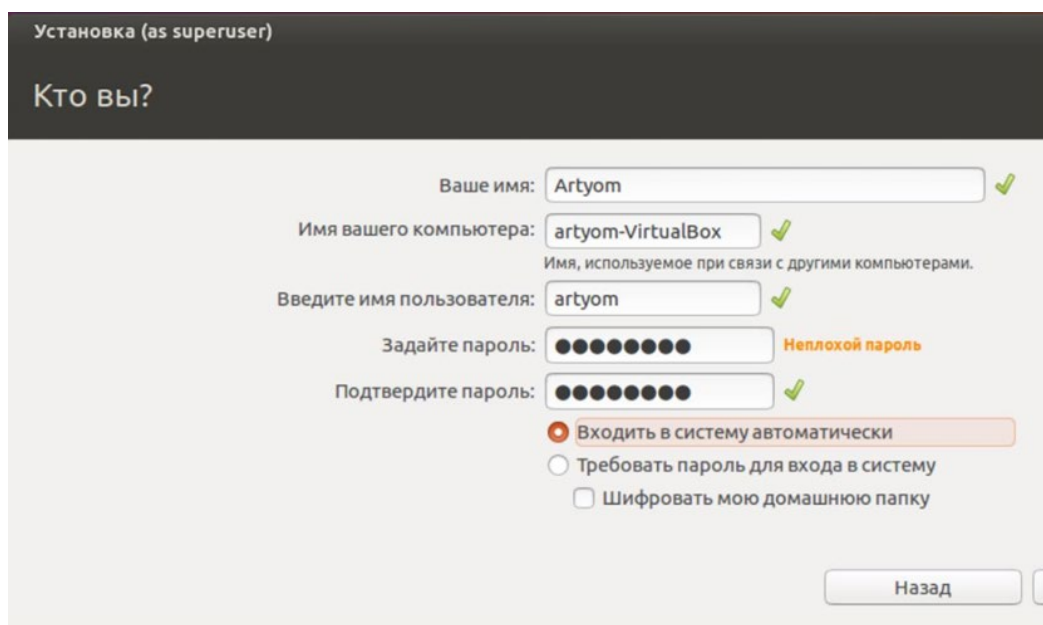


Рисунок 14 – Окно с данными пользователя

Успешно завершили установку ОС Linux Ubuntu на VirtualBox.

1.4 ОС Linux Ubuntu

В рамках данной работы была выбрана операционная система Ubuntu. Данный дистрибутив основан на Debian GNU/Linux и является одним из самых популярных дистрибутивов Linux в мире. Он ориентирован на удобство и простоту использования, включает широко распространённое использование утилиты `sudo`, которая позволяет пользователям выполнять администраторские задачи [3].

Выбор Ubuntu обусловлен также её широкой распространённостью в образовательной среде и среди профессиональных разработчиков. Кроме того, Ubuntu является одним из наиболее документированных дистрибутивов, что значительно упрощает процесс обучения и освоения системы.

К главным особенностям дистрибутива Ubuntu можно отнести:

- 1) стабильность работы – не требует частых перезагрузок компьютера;
- 2) безопасная система, не требующая антивирусов;
- 3) бесплатная операционная система;
- 4) приятный и понятный интерфейс;
- 5) быстрая и простая установка;
- 6) имеется специализированный менеджер пакетов;
- 7) малотребовательная к ресурсам компьютера операционная система;
- 8) модульность.

Операционная система Ubuntu имеет модульную архитектуру и включает в себя восемь основных компонентов, каждый из которых выполняет свою функцию в обеспечении работы системы.

Первым элементом является начальный загрузчик Grub, который отвечает за процесс загрузки операционной системы, позволяет выбирать

нужную ОС при мультизагрузке и задавать параметры командной строки для ядра.

Ядро Linux представляет собой центральный компонент всей системы, обеспечивающий управление процессором, оперативной памятью и всеми устройствами ввода-вывода, а также взаимодействие между аппаратным и программным обеспечением [3].

Важную роль играют демоны – это фоновые процессы, которые запускаются системой автоматически (как правило, во время загрузки) и работают без непосредственного участия пользователя, выполняя различные системные задачи, такие как обслуживание сети, управление печатью или планирование заданий.

Командная оболочка предоставляет интерфейс командной строки, позволяя пользователю управлять системой путём ввода текстовых команд. Вместе с ней поставляются утилиты командной оболочки — набор встроенных команд, необходимых для выполнения базовых операций с файлами, процессами и системными настройками.

Графический интерфейс состоит из 2 компонентов. Графический сервер не входит в состав ядра и представляет собой основу для отображения графической информации. Поверх него работает среда рабочего стола - в случае Ubuntu это среда Unity, которая формирует визуальное представление системы: фон рабочего стола, панели задач, заголовки и границы окон, а также элементы управления.

Наконец, программы рабочего стола – это прикладные приложения, которые запускаются пользователем в графической среде, но не являются её неотъемлемой частью. К ним относятся текстовые редакторы, браузеры, медиаплееры и другие полезные программы.

Каждый из перечисленных компонентов выполняет свою строго определённую функцию. Например, начальный загрузчик Grub отвечает за выбор операционной системы при включении компьютера, если на устройстве установлено несколько ОС. Ядро Linux является связующим звеном между аппаратным обеспечением и программным обеспечением, обеспечивая корректное взаимодействие всех устройств компьютера.

Таким образом, Ubuntu представляет собой современную, безопасную и функциональную операционную систему, которая благодаря своей модульной архитектуре и открытому исходному коду предоставляет пользователю широкие возможности для настройки и адаптации под любые задачи — от повседневного использования до профессиональной разработки программного обеспечения.

2. Установка IDE Geany

2.1 Описание процесса установки

Зашли в терминал. Затем в нем ввели необходимые команды. Изображение терминала представлено на рисунке 15.

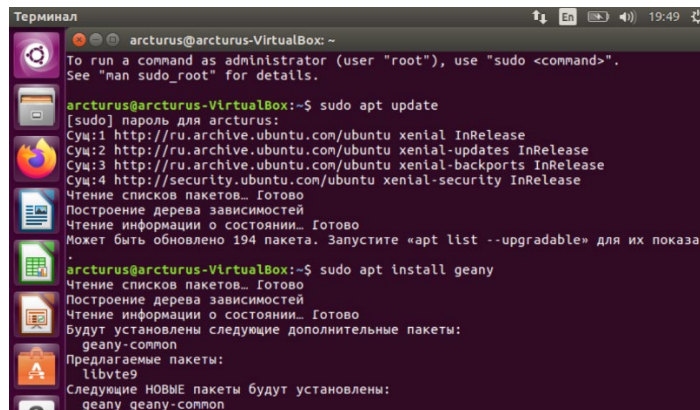


Рисунок 15 – Терминал

Команда `sudo apt update` предназначена для установки обновлений списка пакетов.

Команда `sudo apt install geany` предназначена для установки Geany.

Завершили установку. Затем открыли программу. Изображение интерфейса представлено на рисунке 16.

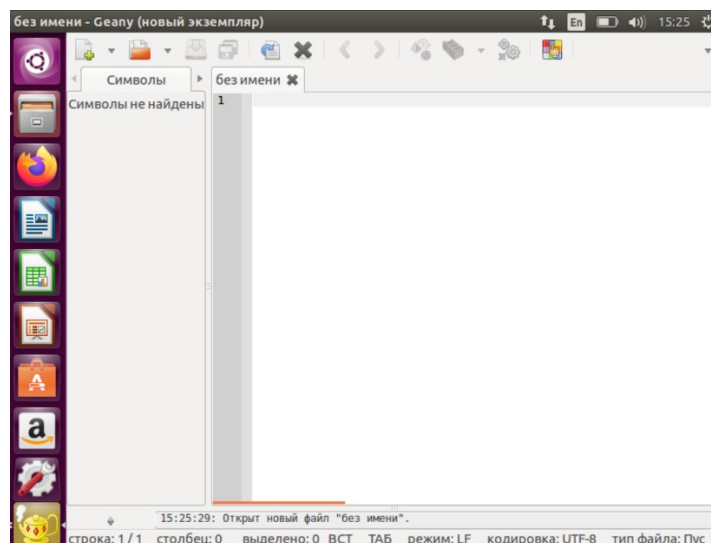


Рисунок 16 – Интерфейс Geany

2.2 Описание IDE Geany

В качестве среды разработки была выбрана Geany – легковесная интегрированная среда разработки. Geany представляет собой кроссплатформенную среду разработки, построенную с использованием библиотеки GTK+. Она распространяется под лицензией GNU General Public License. Несмотря на свою лёгкость и минималистичность, Geany включает все необходимые функции для комфортной работы с кодом на различных языках программирования, включая C, C++, Java, Python, PHP и многие другие [4].

Интерфейс Geany интуитивно понятен и не перегружен лишними элементами. Основное окно содержит редактор кода с подсветкой синтаксиса, панель инструментов для быстрого доступа к часто используемым операциям, встроенный терминал для выполнения команд и панель сообщений для отображения результатов компиляции и отладки. Такая организация рабочего пространства позволяет разработчику сосредоточиться непосредственно на написании кода, не отвлекаясь на сложные настройки и управление многочисленными панелями.

Выбор данной среды разработки для выполнения практической работы обусловлен несколькими весомыми причинами. Во-первых, Geany обладает крайне низкими системными требованиями, что особенно важно при работе на виртуальной машине с ограниченными ресурсами.

Во-вторых, Geany изначально разрабатывалась для Unix-подобных систем, включая Linux, поэтому она прекрасно интегрируется с экосистемой Ubuntu и инструментами разработки GNU. В частности, Geany автоматически обнаруживает установленный компилятор GCC и отладчик GDB, что позволяет начать работу сразу после установки без дополнительной настройки.

В-третьих, Geany предоставляет все необходимые функции для эффективной разработки. Подсветка синтаксиса делает код более читаемым и помогает быстро находить ошибки. Автодополнение кода ускоряет написание программ и снижает количество опечаток.

Таким образом, Geany представляет собой оптимальное решение для разработки программы на языке C++ в среде Ubuntu на виртуальной машине.

3. Разработка программы на языке C++ в среде Geany

3.1 Требования к программным средствам

Сортировка Шелла – простейшее расширение метода вставок, быстроедействие которого выше за счет обеспечения возможности обмена местами элементов, которые находятся далеко один от другого. Основная идея этого алгоритма заключается в том, что вначале устраняют массовый беспорядок в массиве, сравнивая и переставляя далеко стоящие друг от друга элементы. Далее интервал между сравниваемыми элементами постепенно уменьшается и, в конце концов, становится равным единице. Как показывают оценки, алгоритм сортировки Шелла имеет сложность $O(n \log n)$, что значительно лучше рассмотренных выше алгоритмов.

Алгоритм шага Кнута[5] определяет интервалы сравнения элементов в списке. Он основан на последовательности чисел, называемых последовательностью Кнута, которая вычисляется с использованием формулы: $gap = 3 * gap + 1$.

Кнут провел множество экспериментов и предложил следующую формулу выбора шагов (h) для массива длины N. В последовательности $h_1 = 1$, $h_{s+1} = 3 * h_s + 1$, ... взять h_i такое, что $h_{i+2} \geq N$.

Алгоритм шага Вирта[5] для сортировки Шелла заключается в следующем:

1. Определите последовательность шагов Вирта.

Эффективность алгоритма во многом зависит от выбора последовательности $h_1, h_2, \dots, h_t$ ($h_t=1, h_{i+1} < h_i$).

Н. Вирт предлагает следующую формулу для вычисления количества проходов t и последовательности шагов $h_1, \dots, h_t$:

$$t = \lfloor \log_2 n \rfloor - 1, h_t = 1, h_{k-1} = 2h_k + 1,$$

которая дает следующие значения $ht=1$, $ht-1=3$, $ht-2=7$, $ht-3=15$ и т.д.

2. Пока текущий шаг h больше нуля, выполняйте следующие действия:

- разделите массив на подмассивы шириной h ;
- для каждого подмассива выполняйте сортировку вставками;
- уменьшайте текущий шаг h до $h = (h - 1) / 2$.

3. По окончании всех шагов Вирта массив будет отсортирован.

Алгоритм шага Вирта для сортировки Шелла основан на идее предварительной сортировки массива при больших интервалах, а затем постепенном сужении интервалов до сортировки отдельных элементов массива. Это позволяет улучшить эффективность сортировки Шелла по сравнению с сортировкой вставками, особенно для больших массивов.

Алгоритм шага деления пополам[5] предлагает разделить текущий шаг на 2 на каждой итерации.

Процесс работы алгоритма шага деления пополам выглядит следующим образом:

1. Выбирается начальный шаг, который обычно равен половине размера массива.

2. Пока шаг больше нуля, выполняются следующие действия:

- производится сортировка элементов на расстоянии шага с использованием вставки или другого выбранного алгоритма сортировки;
- шаг уменьшается вдвое;
- продолжают сортировки с новым шагом.

3.2 Требования к программным средствам

Определили следующие функциональные требования:

- 1) работа с файлами (чтение и запись);

- 2) выполнение сортировки массива;
- 3) подсчет времени сортировки разными модификациями алгоритма Шелла.

Составили диаграмму вариантов использования, отражающую взаимодействие пользователя и системы. Диаграмма представлена на рисунке 17.



Рисунок 16 – Диаграмма вариантов использования

Для алгоритма сортировки массива методом Шелла, определение шага делением пополам была разработана диаграмма деятельности. Диаграмма представлена на рисунке 17.

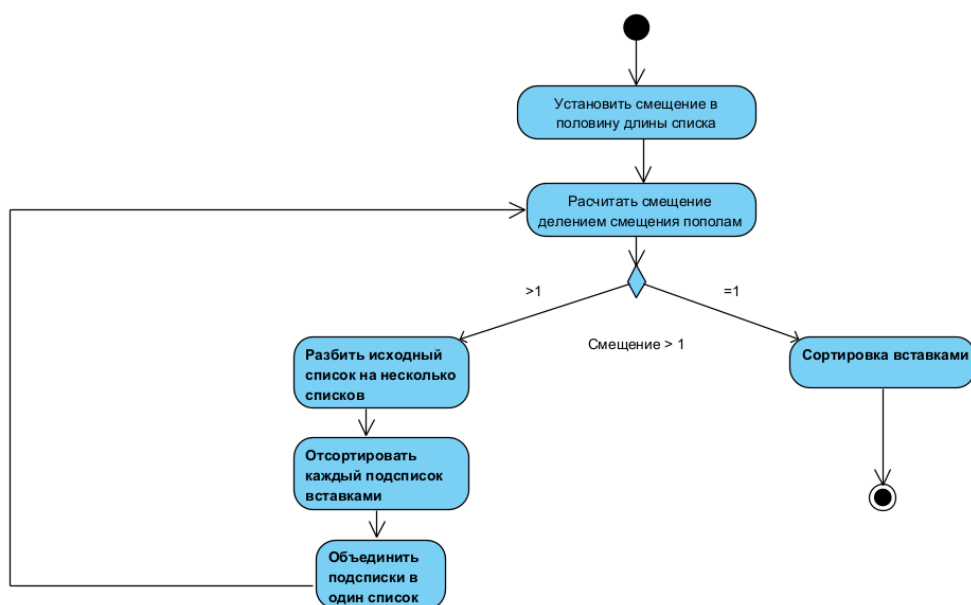


Рисунок 17 – Диаграмма последовательности действия

Как видно из диаграммы, первоначально в методе устанавливается значение смещения. Далее идёт условие, которое проверяет значение смещения. Далее исходный список разбивается на несколько подсписков согласно размеру смещения. Затем каждый подсписок сортируется вставками. Отсортированные подсписки снова объединяются в один, происходит перерасчет смещения. Если смещение больше 1, то разбиение на подсписки повторяется, а если равно 1, то весь список сортируется вставками. После этого программа выводит результат.

3.3 Функциональное тестирование

В работе было выполнено функциональное тестирование [1] разработанного программного обеспечения. Результаты тестирования приведены в таблице 1.

Таблица 1 – функциональное тестирование разработанного ПО

Состав теста	Ожидаемый результат	Наблюдаемый результат
Тестирование сортировки Шелла по введенным данным. (Размер массива – 1000)	Должна быть выполнена сортировка. Также должны быть отображены результаты.	Выполнена сортировка по введенным данным, были выведены данные и записаны в файлы (рисунок 18).

Тестирование сортировки Шелла по введенным данным. (Размер массива – 10000)	Должна быть выполнена сортировка. Также должны быть отображены результаты.	Выполнена сортировка по введенным данным, были выведены данные и записаны в файлы (рисунок 19).
Тестирование сортировки Шелла по введенным данным. (Размер массива – 50000)	Должна быть выполнена сортировка. Также должны быть отображены результаты.	Выполнена сортировка по введенным данным, были выведены данные и записаны в файлы (рисунок 20).
Тестирование сортировки Шелла по введенным данным. (Размер массива – 100000)	Должна быть выполнена сортировка. Также должны быть отображены результаты.	Выполнена сортировка по введенным данным, были выведены данные и записаны в файлы (рисунок 21).
Тестирование сортировки Шелла по введенным данным. (Размер массива – 1000000)	Должно быть выведено сообщение об ошибке.	Было выведено сообщение об ошибке (рисунок 22).
Тестирование сортировки Шелла (чтение из файла) (Размер массива – 10)	Должна быть выполнена сортировка. Также должны быть отображены результаты.	Выполнена сортировка по введенным данным, были выведены данные и записаны в файлы (рисунок 23).
Тестирование устойчивости программы. Ввели некорректный номер действия.	Должно быть выведено сообщение об ошибке.	Было выведено сообщение об ошибке (рисунок 24).
Тестирование устойчивости программы. Ввели некорректные данные в файл.	Должно быть выведено сообщение об ошибке.	Было выведено сообщение об ошибке (рисунок 25).

Результаты работы программы представлены на рисунках 18-25.

```

arcturus@arcturus-VirtualBox: ~/Рабочий стол/ShellSort
536 538 539 539 540 540 541 542 543 544 545 546 551 551 553 555 555 557 557 557
557 559 560 562 565 566 566 567 569 569 570 570 570 571 572 573 574 576 576 578
578 578 579 579 580 580 581 584 584 590 591 592 593 593 596 597 597 598 599 601
603 603 606 606 607 609 610 610 611 614 615 617 617 618 619 619 619 619 620 620
621 624 626 627 628 628 629 629 632 632 635 635 635 636 636 638 638 640 640 641
641 643 643 643 643 644 644 645 645 645 646 646 648 649 651 651 653 655 656 657
660 660 661 661 662 662 663 664 666 666 667 668 668 668 669 669 670 671 672 673
676 676 676 678 679 680 680 682 683 684 685 685 686 688 688 688 688 689 690 690
690 692 693 693 694 694 696 697 698 699 699 700 701 704 706 706 706 707 709 710
711 711 711 714 715 718 720 720 721 722 722 722 724 726 727 729 731 734 735 735
736 736 737 738 739 739 740 741 743 745 749 749 752 752 752 756 756 756 757 757
759 760 760 760 761 762 763 763 766 766 769 769 770 771 771 773 773 776 778 780
781 781 781 782 783 784 785 785 785 791 791 793 793 794 795 798 801 803 805 806
807 807 807 809 809 810 812 813 813 814 815 816 819 819 820 821 822 822 822 823
828 828 829 829 829 833 833 833 834 838 842 842 843 846 847 847 847 848 849 850
851 852 852 852 854 854 856 856 856 856 857 857 857 858 858 859 860 861 862 863
864 865 865 865 866 867 868 869 869 871 873 874 876 878 879 881 882 884 885 885
885 888 891 891 894 895 897 898 899 899 899 902 902 902 903 903 904 906 907 908
911 912 913 916 917 918 918 919 920 921 921 925 926 926 928 932 934 934 934 935
935 935 936 936 937 937 938 938 939 939 940 940 940 943 944 948 948 949 949 949
950 951 953 954 954 955 956 956 957 957 958 962 963 963 965 966 966 967 967 968
968 969 970 970 972 973 974 975 975 976 977 977 977 978 979 979 980 980 983 983
986 987 989 991 991 991 991 994 995 995 997 997 997 997 998 998 998

Время сортировки:
Метод Кнута: 127 тиков
Метод Вирта: 107 тиков
Метод деления пополам: 125 тиков
Запись результатов в файл output.txt...
Результаты успешно записаны в output.txt
Нажмите Enter для выхода...

```

Рисунок 18 – Сортировка массива из 1000 элементов

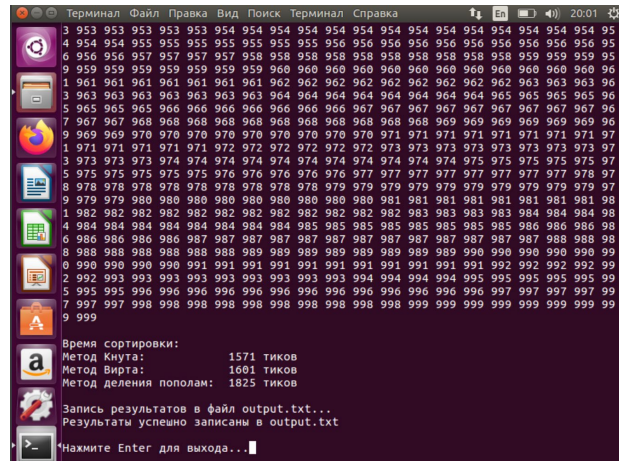


Рисунок 19 – Сортировка массива из 10000 элементов

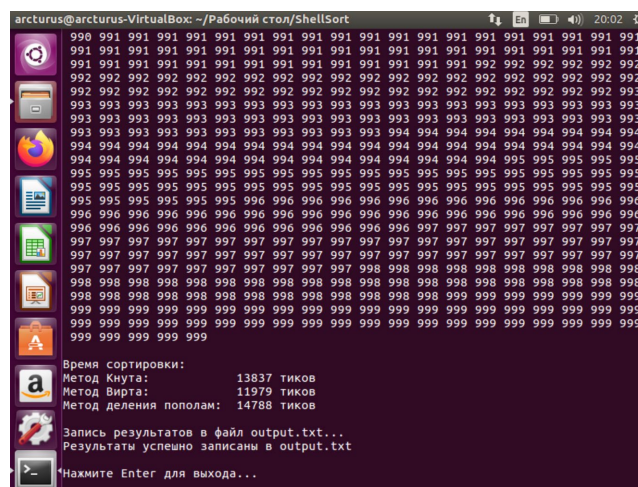


Рисунок 20 – Сортировка массива из 50000 элементов

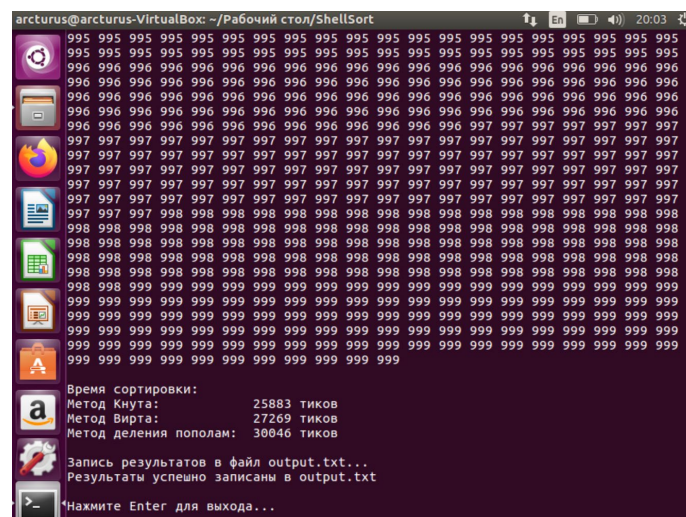


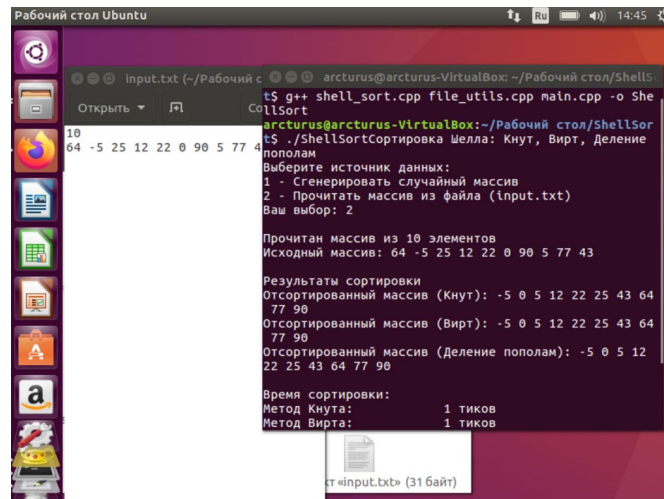
Рисунок 21 – Сортировка массива из 100000 элементов

```

arcturus@arcturus-VirtualBox:~/Рабочий стол/ShellSort$ ./ShellSort
Сортировка Шелла: Кнут, Вирт, Деление пополам
Выберите источник данных:
1 - Сгенерировать случайный массив
2 - Прочитать массив из файла (input.txt)
Ваш выбор: 1
Введите размер массива: 1000000
Размер должен быть от 1 до 100000! Введите заново:

```

Рисунок 22 – Попытка ввести некорректное число



```

Рабочий стол Ubuntu
Input.txt (~/Рабочий с
arcturus@arcturus-VirtualBox: ~/Рабочий стол/ShellSort$ g++ shell_sort.cpp file_utils.cpp main.cpp -o ShellSort
arcturus@arcturus-VirtualBox: ~/Рабочий стол/ShellSort$ ./ShellSort
Сортировка Шелла: Кнут, Вирт, Деление пополам
Выберите источник данных:
1 - Сгенерировать случайный массив
2 - Прочитать массив из файла (input.txt)
Ваш выбор: 2
Прочитан массив из 10 элементов
Исходный массив: 64 -5 25 12 22 0 90 5 77 43
Результаты сортировки
Отсортированный массив (Кнут): -5 0 5 12 22 25 43 64 77 90
Отсортированный массив (Вирт): -5 0 5 12 22 25 43 64 77 90
Отсортированный массив (Деление пополам): -5 0 5 12 22 25 43 64 77 90
Время сортировки:
Метод Кнута: 1 тиков
Метод Вирта: 1 тиков
Файл input.txt (31 байт)

```

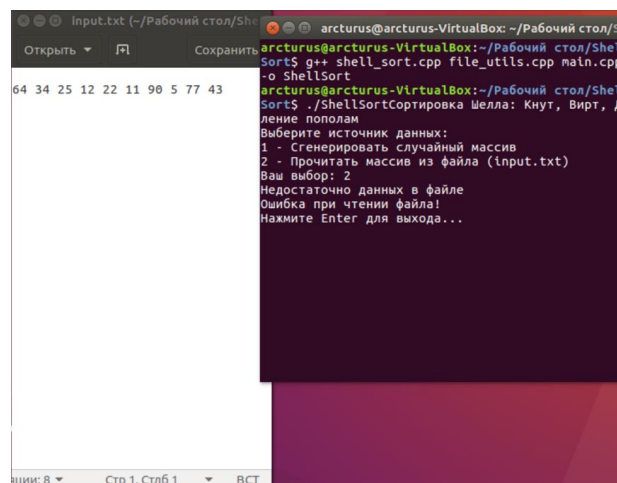
Рисунок 23 – Чтение из файла и запись в другой файл

```

arcturus@arcturus-VirtualBox:~/Рабочий стол/ShellSort$ ./ShellSort
Сортировка Шелла: Кнут, Вирт, Деление пополам
Выберите источник данных:
1 - Сгенерировать случайный массив
2 - Прочитать массив из файла (input.txt)
Ваш выбор: 3
Ошибка! Введите 1 или 2:

```

Рисунок 24 – Попытка ввода некорректного номера операции



```

Input.txt (~/Рабочий стол/ShellSort)
arcturus@arcturus-VirtualBox: ~/Рабочий стол/ShellSort$ g++ shell_sort.cpp file_utils.cpp main.cpp -o ShellSort
arcturus@arcturus-VirtualBox: ~/Рабочий стол/ShellSort$ ./ShellSort
Сортировка Шелла: Кнут, Вирт, Деление пополам
Выберите источник данных:
1 - Сгенерировать случайный массив
2 - Прочитать массив из файла (input.txt)
Ваш выбор: 2
Недостаточно данных в файле
Ошибка при чтении файла!
Нажмите Enter для выхода...

```

Рисунок 25 – Попытка чтения файла с некорректными данными

В ходе выполнения тестирования несовпадения ожидаемого и наблюдаемого результата не выявлены. Следовательно, можно сделать вывод, что программа работает корректно.

4. Анализ результатов

Были составлены таблица и график на основе данных, полученных из предыдущего раздела. Изображения таблицы и графика представлены на рисунках 26 и 27, соответственно.

Алгоритм	Время (тики)			
	1000	10000	50000	100000
Кнут	127	1571	13837	25883
Вирт	107	1601	11979	27269
Деление пополам	125	1825	14788	30046

Рисунок 26 – Сравнительная таблица результатов

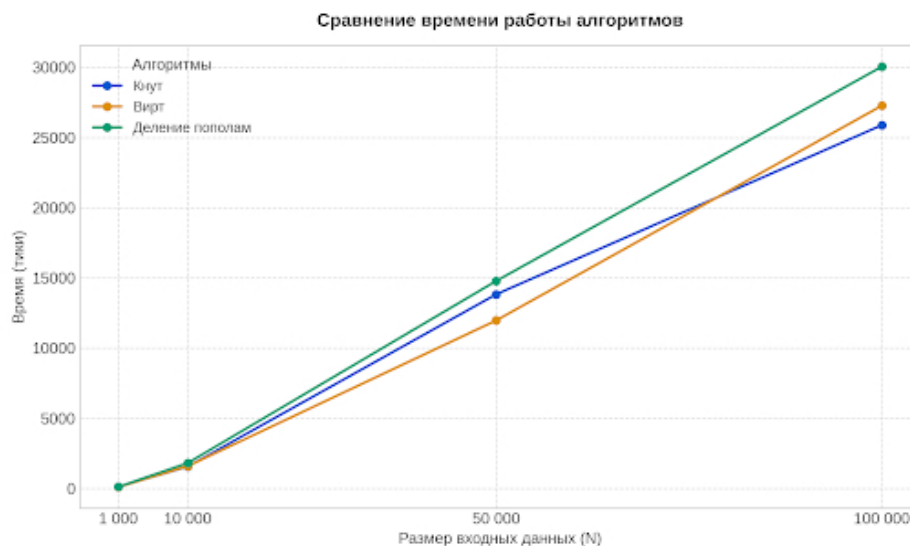


Рисунок 27 – График

По таблице можно сделать вывод, что во всех случаях хуже всего проявил себя алгоритм делением пополам. Когда как Кнут и Вирт показали себя примерно одинаково. Скорость сортировки зависит также и от степени отсортированности исходного массива [5]. Можно утверждать, что могут быть случаи, когда случайно созданный массив уже будет полностью упорядочен, либо отсортирован в обратном порядке, что также будет влиять на скорость его сортировки разными модификациям.

Заключение

В ходе выполнения практической работы была изучена и установлена операционная система Ubuntu на виртуальную машину VirtualBox, установлена интегрированная среда разработки Geany в ОС Ubuntu, были изучены основные инструменты среды и разработана программа на языке C++. Результаты испытаний показали, что программа работает согласно формулировке задачи.

Список используемой литературы

1. Виды тестирования программного обеспечения. Режим доступа: <http://www.protesting.ru/testing/testtypes.html> (Дата обращения: 20.06.2026)
2. VirtualBox. Учебно-методическое обеспечение дисциплины (Дата обращения: 19.06.2026)
3. Linux Ubuntu. Учебно-методическое обеспечение дисциплины (Дата обращения: 19.06.2026)
4. Geany. Режим доступа: <https://pingvinus.ru/note/geany-tune> (Дата обращения: 20.06.2026)
5. Р.Лафоре. Объектно-ориентированное программирование в C++. - Питер, 2004. - 923с (Дата обращения: 19.06.2026)
Ссылка на GitHub: https://github.com/aaaartemkaa/SummerPractic_24VVV2_5V

Приложение А

Листинг программы

main.cpp

```
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <limits>
#include "shell_sort.h"
#include "file_utils.h"
using namespace std;

int main()
{
    setlocale(LC_ALL, "Russian");
    srand(time(0));
    int* original = 0;
    int size;
    int choice;
    cout << "Сортировка Шелла: Кнут, Вирт, Деление пополам" << endl;
    cout << "Выберите источник данных:" << endl;
    cout << "1 - Сгенерировать случайный массив" << endl;
    cout << "2 - Прочитать массив из файла (input.txt)" << endl;
    cout << "Ваш выбор: ";
    cin >> choice;
    while (cin.fail() || (choice != 1 && choice != 2))
    {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        cout << "Ошибка! Введите 1 или 2: ";
        cin >> choice;
    }
    if (choice == 1)
    {
        cout << "Введите размер массива: ";
        cin >> size;
        while (cin.fail() || size <= 0 || size > 100000)
        {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Размер должен быть от 1 до 100000! Введите заново: ";
            cin >> size;
        }
        original = new int[size];
        for (int i = 0; i < size; i++)
        {
            original[i] = rand() % 1000;
        }
        cout << endl << "Сгенерирован массив из " << size << " элементов" << endl;
    }
    else if (choice == 2)
    {
        if (!readArrayFromFile("input.txt", original, size))
        {
            cout << "Ошибка при чтении файла!" << endl;
            cout << "Нажмите Enter для выхода...";
            cin.ignore();
            cin.get();
            return 1;
        }
    }
}
```



```

        cout << endl << "Прочитан массив из " << size << " элементов" << endl;
    }
    cout << "Исходный массив: ";
    printArray(original, size);
    cout << endl;
    int* arrKnuth = new int[size];
    int* arrVirt = new int[size];
    int* arrDividing = new int[size];
    copyArray(original, arrKnuth, size);
    copyArray(original, arrVirt, size);
    copyArray(original, arrDividing, size);
    long long timeKnuth = measureTime(shellSortKnuth, arrKnuth, size);
    long long timeVirt = measureTime(shellSortVirt, arrVirt, size);
    long long timeDividing = measureTime(shellSortDividingInHalf, arrDividing,
size);
    cout << "Результаты сортировки" << endl;
    cout << "Отсортированный массив (Кнут): ";
    printArray(arrKnuth, size);
    cout << "Отсортированный массив (Вирт): ";
    printArray(arrVirt, size);
    cout << "Отсортированный массив (Деление пополам): ";
    printArray(arrDividing, size);
    cout << endl;
    cout << "Время сортировки:" << endl;
    cout << "Метод Кнута: " << timeKnuth << " тиков" << endl;
    cout << "Метод Вирта: " << timeVirt << " тиков" << endl;
    cout << "Метод деления пополам: " << timeDividing << " тиков" << endl;
    cout << endl << "Запись результатов в файл output.txt..." << endl;
    if (writeResultsToFile("output.txt", arrKnuth, size, timeKnuth, timeVirt,
timeDividing))
    {
        cout << "Результаты успешно записаны в output.txt" << endl;
    }
    writeArrayToFile("sorted_knuth.txt", arrKnuth, size, "Сортировка Шелла
(Кнут):");
    writeArrayToFile("sorted_virt.txt", arrVirt, size, "Сортировка Шелла (Вирт):");
    writeArrayToFile("sorted_dividing.txt", arrDividing, size, "Сортировка Шелла
(Деление пополам):");
    delete[] original;
    delete[] arrKnuth;
    delete[] arrVirt;
    delete[] arrDividing;
    cout << "\nНажмите Enter для выхода...";
    cin.ignore();
    cin.get();
    return 0;
}

```

file_utils.cpp

```

#include "file_utils.h"
#include <fstream>
#include <iostream>
using namespace std;

bool readArrayFromFile(const string& filename, int*& arr, int& n)
{
    ifstream file(filename.c_str());
    if (!file.is_open())
    {
        cerr << "Не удалось открыть файл " << filename << endl;
        return false;
    }
}

```

```

    }
    if (!(file >> n))
    {
        cerr << "Файл пуст или не содержит размера" << endl;
        file.close();
        return false;
    }
    if (n <= 0 || n > 100000)
    {
        cerr << "Неверный размер массива (" << n << ")" << endl;
        file.close();
        return false;
    }
    arr = new int[n];
    for (int i = 0; i < n; i++)
    {
        if (!(file >> arr[i]))
        {
            cerr << "Недостаточно данных в файле" << endl;
            delete[] arr;
            arr = 0;
            file.close();
            return false;
        }
    }
    file.close();
    return true;
}

bool writeResultsToFile(const string& filename, int arr[], int n,
    long long timeKnuth, long long timeVirt, long long timeDividing)
{
    ofstream file(filename.c_str());
    if (!file.is_open())
    {
        cerr << "Не удалось создать файл " << filename << endl;
        return false;
    }
    file << "Результаты сортировки" << endl;
    file << "Размер массива: " << n << " элементов" << endl << endl;
    file << "Отсортированный массив:" << endl;
    for (int i = 0; i < n; i++)
    {
        file << arr[i] << " ";
        if ((i + 1) % 20 == 0) file << endl;
    }
    file << endl << endl;
    file << "Время сортировки" << endl;
    file << "Метод Кнута: " << timeKnuth << " тиков" << endl;
    file << "Метод Вирта: " << timeVirt << " тиков" << endl;
    file << "Метод деления пополам: " << timeDividing << " тиков" << endl;
    file.close();
    return true;
}

void writeArrayToFile(const string& filename, int arr[], int n, const string& label)
{
    ofstream file(filename.c_str());
    if (!file.is_open()) return;
    file << label << endl;
    for (int i = 0; i < n; i++)
    {
        file << arr[i] << " ";
        if ((i + 1) % 20 == 0) file << endl;
    }
}

```

```

    }
    file << endl;
    file.close();
}

```

shell_sort.cpp

```

#include "shell_sort.h"
#include <iostream>
#include <ctime>
using namespace std;

void shellSortKnuth(int arr[], int n)
{
    int gap = 1;
    while (gap < n / 3)
    {
        gap = 3 * gap + 1;
    }
    while (gap >= 1)
    {
        for (int i = gap; i < n; i++)
        {
            int temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap)
            {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
        gap = gap / 3;
    }
}

void shellSortVirt(int arr[], int n)
{
    int gap = 1;
    while (gap < n / 2)
    {
        gap = 2 * gap + 1;
    }
    while (gap >= 1)
    {
        for (int i = gap; i < n; i++)
        {
            int temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap)
            {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
        gap = (gap - 1) / 2;
    }
}

void shellSortDividingInHalf(int arr[], int n)
{
    int gap = n / 2;
    while (gap >= 1)

```

```

    {
        for (int i = gap; i < n; i++)
        {
            int temp = arr[i];
            int j;
            for (j = i; j >= gap && arr[j - gap] > temp; j -= gap)
            {
                arr[j] = arr[j - gap];
            }
            arr[j] = temp;
        }
        gap = gap / 2;
    }
}

void printArray(int arr[], int n)
{
    for (int i = 0; i < n; i++)
    {
        cout << arr[i] << " ";
    }
    cout << endl;
}

void copyArray(int source[], int dest[], int n)
{
    for (int i = 0; i < n; i++)
    {
        dest[i] = source[i];
    }
}

long long measureTime(void (*sortFunc)(int[], int), int arr[], int n)
{
    clock_t start = clock();
    sortFunc(arr, n);
    clock_t end = clock();
    return (long long)(end - start);
}

```

shell_sort.h

```

#ifndef SHELL_SORT_H
#define SHELL_SORT_H
void shellSortKnuth(int arr[], int n);
void shellSortVirt(int arr[], int n);
void shellSortDividingInHalf(int arr[], int n);
void printArray(int arr[], int n);
void copyArray(int source[], int dest[], int n);
long long measureTime(void (*sortFunc)(int[], int), int arr[], int n);
#endif

```

file_utils.h

```

#ifndef FILE_UTILS_H
#define FILE_UTILS_H
#include <string>
bool readArrayFromFile(const std::string& filename, int*& arr, int& n);
bool writeResultsToFile(const std::string& filename, int arr[], int n,
    long long timeKnuth, long long timeVirt, long long timeDividing);
void writeArrayToFile(const std::string& filename, int arr[], int n, const
    std::string& label);
#endif

```